In [1]:
```
import nltk
import pandas as pd
```

In [2]:
```
df=pd.read_csv("spam.csv",encoding="latin-1")
df.head()
```

Out[2]:

	v1	v2	Unnamed: 2	Unnamed: 3	Unnamed: 4
0	ham	Go until jurong point, crazy.. Available only ...	NaN	NaN	NaN
1	ham	Ok lar... Joking wif u oni...	NaN	NaN	NaN
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...	NaN	NaN	NaN
3	ham	U dun say so early hor... U c already then say...	NaN	NaN	NaN
4	ham	Nah I don't think he goes to usf, he lives aro...	NaN	NaN	NaN

In [3]:
```
df.drop(["Unnamed: 2","Unnamed: 3","Unnamed: 4"],axis=1,inplace=True)
```

In [4]:
```
df.head()
```

Out[4]:

	v1	v2
0	ham	Go until jurong point, crazy.. Available only ...
1	ham	Ok lar... Joking wif u oni...
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...
3	ham	U dun say so early hor... U c already then say...
4	ham	Nah I don't think he goes to usf, he lives aro...

In [5]:
```
df.columns=["label","text"]
```

In [6]:
```
df.head()
```

Out[6]:

	label	text
0	ham	Go until jurong point, crazy.. Available only ...
1	ham	Ok lar... Joking wif u oni...
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...
3	ham	U dun say so early hor... U c already then say...
4	ham	Nah I don't think he goes to usf, he lives aro...

In [7]:
```
nltk.download("stopwords")
```

```
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\vipul\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
```

Out[7]: True

In [8]:
```
import nltk
import string
import re
stopwords=nltk.corpus.stopwords.words("english")
```

In [9]:
```
def clean_text(text):
    text="".join([char.lower() for char in text if char not in string.punctuat
    tokens=re.split("\W+",text)
    text=[word for word in tokens if word not in stopwords]
    return text
```

In [10]:
```
x="Nah I don't think he goes to usf, he lives aro?"
x="".join([char.lower() for char in x if char not in string.punctuation])
x
```

Out[10]: 'nah i dont think he goes to usf he lives aro'

In [11]:
```
tokens=re.split("\W+",x)
```

In [12]:
```
tokens
```

Out[12]: ['nah', 'i', 'dont', 'think', 'he', 'goes', 'to', 'usf', 'he', 'lives', 'ar
o']

In [13]:
```
clean_text("Nah I don't think he goes to usf, he lives aro?")
```

Out[13]: ['nah', 'dont', 'think', 'goes', 'usf', 'lives', 'aro']

In [14]:
```
from sklearn.feature_extraction.text import CountVectorizer
count_vect = CountVectorizer(analyzer=clean_text)
X_count=count_vect.fit_transform(df["text"])
print(X_count.shape)
print(count_vect.get_feature_names_out())
```

```
(5572, 9395)
['' '0' '008704050406' ... 'ûïharry' 'ûò' 'ûówell']
```

In [15]:
```
X_features=pd.DataFrame(X_count.toarray())
X_features.head()
```

Out[15]:

	0	1	2	3	4	5	6	7	8	9	...	9385	9386	9387	9388	9389	9390	9391
0	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0
2	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0
3	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0
4	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0

5 rows × 9395 columns

In [16]:
```
from sklearn.ensemble import RandomForestClassifier
```

In [17]:
```
from sklearn.model_selection import train_test_split
```

In [18]:
```
X_train,X_test,y_train,y_test=train_test_split(X_features,df["label"],test_siz
```

In [19]:
```
rf=RandomForestClassifier()
rf_model=rf.fit(X_train,y_train)
```

In [20]:
```
y_pred=rf_model.predict(X_test)
```

In [21]:
```
from sklearn.metrics import classification_report
```

In [22]:
```
print(classification_report(y_test,y_pred))
```

```
              precision    recall  f1-score   support

         ham       0.97      1.00      0.98       953
        spam       1.00      0.79      0.88       162

    accuracy                           0.97      1115
   macro avg       0.98      0.90      0.93      1115
weighted avg       0.97      0.97      0.97      1115
```

In [23]:
```
text=["I Kill You"]
count_vect1=count_vect.transform(text)
X_features=pd.DataFrame(count_vect1.toarray())
X_features.head()
```

Out[23]:

	0	1	2	3	4	5	6	7	8	9	...	9385	9386	9387	9388	9389	9390	9391
0	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0

1 rows × 9395 columns

In [24]:
```
y_pred=rf_model.predict(count_vect1)
```

In [25]:
```
y_pred
```

Out[25]:
```
array(['ham'], dtype=object)
```

In [32]:
```
from sklearn.feature_extraction.text import TfidfVectorizer
tfidf_vect = TfidfVectorizer(analyzer=clean_text)
X_tfidf=tfidf_vect.fit_transform(df["text"])
print(X_tfidf.shape)
print(tfidf_vect.get_feature_names_out())
```

```
(5572, 9395)
['' '0' '008704050406' ... 'ûïharry' 'ûò' 'ûówell']
```

In [33]:
```
X_features=pd.DataFrame(X_tfidf.toarray())
X_features.head()
```

Out[33]:

	0	1	2	3	4	5	6	7	8	9	...	9385	9386	9387	9388	938
0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0
1	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0
2	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0
3	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0
4	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0

5 rows × 9395 columns

In [34]:
```
from sklearn.ensemble import RandomForestClassifier
```

In [35]:
```
from sklearn.model_selection import train_test_split
```

In [36]:
```
X_train,X_test,y_train,y_test=train_test_split(X_features,df["label"],test_siz
```

In [38]:
```
rf=RandomForestClassifier()
rf_model=rf.fit(X_train,y_train)
```

In [39]:
```
y_pred=rf_model.predict(X_test)
```

In [40]:
```
from sklearn.metrics import classification_report
```

In [41]:
```
print(classification_report(y_test,y_pred))
```

```
                precision    recall  f1-score   support

         ham       0.97      1.00      0.98       959
        spam       1.00      0.79      0.88       156

    accuracy                           0.97      1115
   macro avg       0.98      0.89      0.93      1115
weighted avg       0.97      0.97      0.97      1115
```

In [42]:
```
text=["I Kill You"]
tfidf_vect1=tfidf_vect.transform(text)
X_features=pd.DataFrame(tfidf_vect1.toarray())
X_features.head()
```

Out[42]:

	0	1	2	3	4	5	6	7	8	9	...	9385	9386	9387	9388	938
0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0

1 rows × 9395 columns

In [43]:
```
y_pred=rf_model.predict(tfidf_vect1)
```

In [44]:
```
y_pred
```

Out[44]:
```
array(['ham'], dtype=object)
```

In [ ]:
```
```